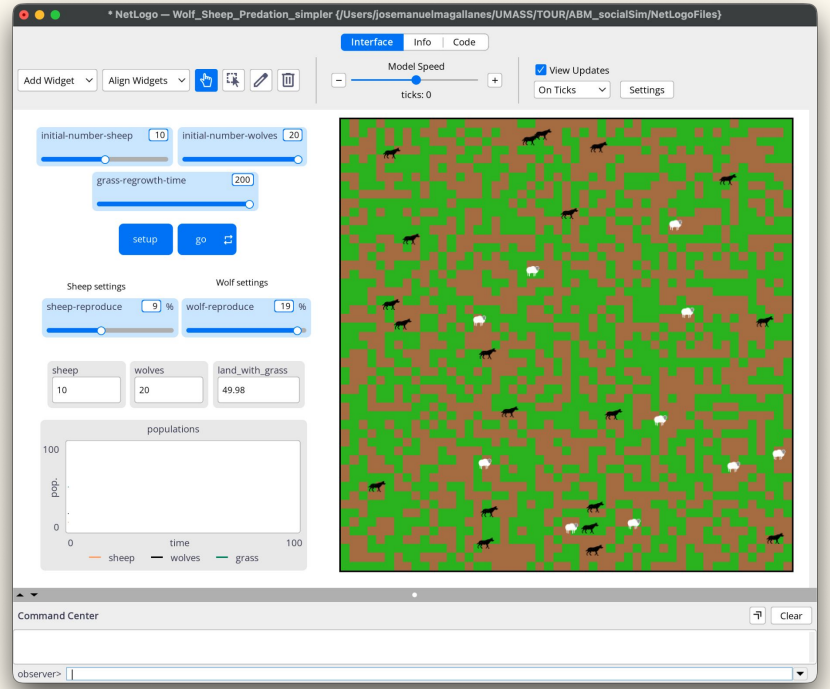
# Scenarios

**Use code:**
**Wolf_Sheep_Predation_simpler.nlogox**

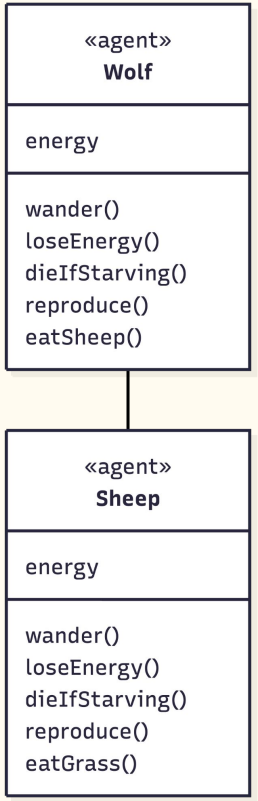
# The case: Wolf and Sheep




## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

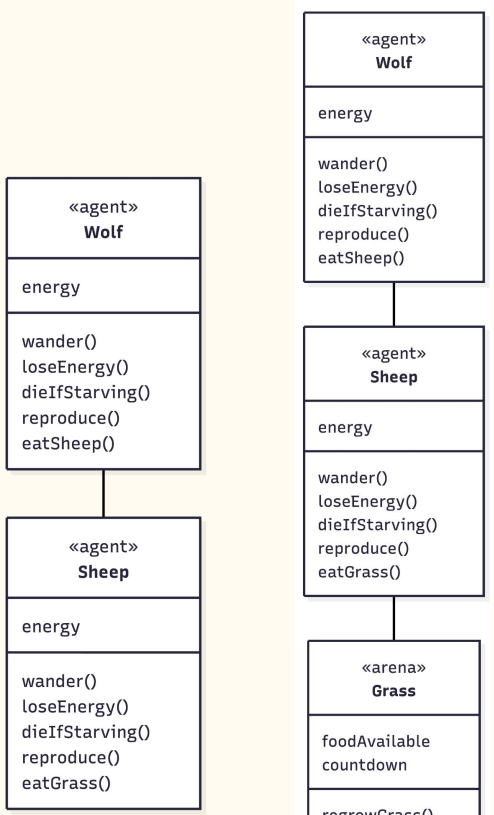
# The case: Wolf and Sheep




## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

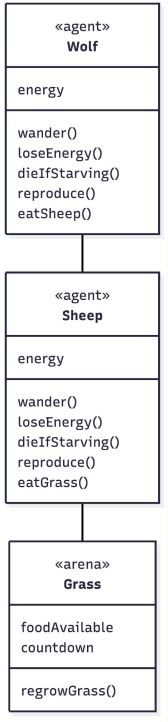
# The case: Wolf and Sheep

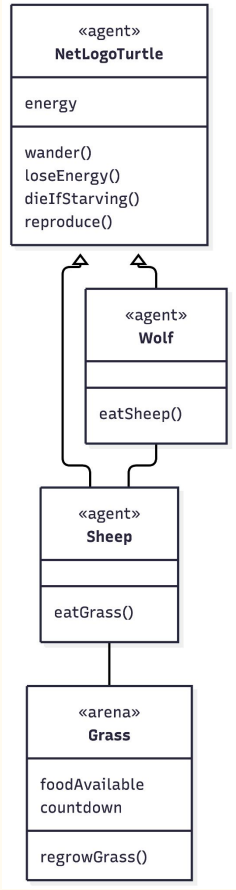



## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

# The case: Wolf and Sheep

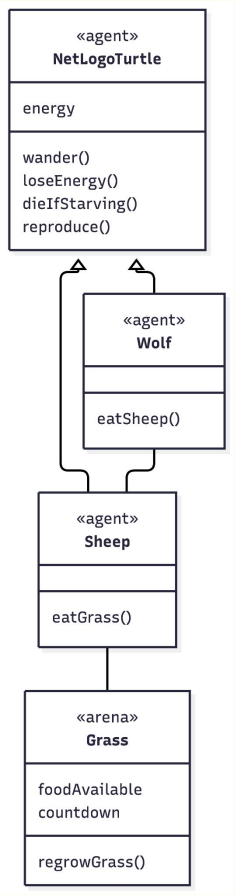


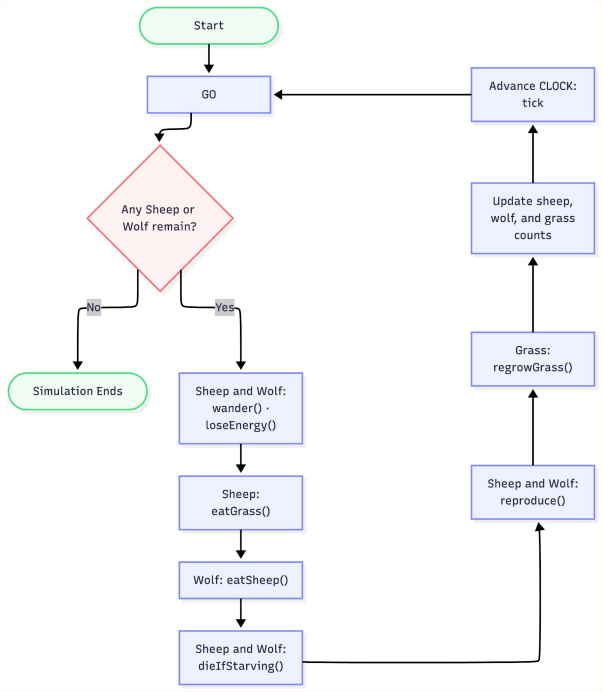




## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

# The case: Wolf and Sheep




## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

# The case: Wolf and Sheep

```
to go
  ; Stop when both animal populations are extinct.
  if not any? turtles [ stop ]

  ask sheep [
    wander
    lose-energy
    eat-grass
    die-if-starving
    reproduce sheep-reproduce-probability
  ]

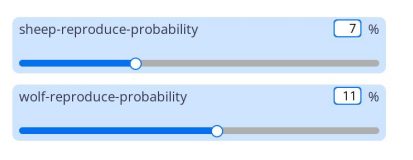
  ask wolves [
    wander
    lose-energy
    eat-sheep
    die-if-starving
    reproduce wolf-reproduce-probability
  ]

  ask patches [
    regrow-grass
  ]

  set current_grass count patches with [food-available?]
  tick
end
```

## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.



# The case: Wolf and Sheep

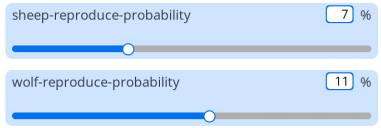
```
to wander
  rt random 360
  fd 1
end

to lose-energy
  set energy energy - 1
end

to die-if-starving
  ; Animals with zero energy survive this check.
  if energy < 0 [ die ]
end

to reproduce [ reproduction-probability ]
  ; Probability is expressed as a percentage from 0 to 100.
  if random-float 100 < reproduction-probability [
    set energy energy / 2

    ; Offspring inherit the parent's breed and halved energy.
    hatch 1 [
      rt random-float 360
      fd 1
    ]
  ]
end
```




## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

**Common procedures**



# The case: Wolf and Sheep

**SHEEP procedures**



## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

```
to eat-grass
  if food-available? [
    set food-available? false
    set pcolor brown
    set energy energy + sheep-gain-from-food
  ]
end
```

# The case: Wolf and Sheep

**WOLVES procedures**

## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

```
to eat-sheep
  let prey one-of sheep-here

  if prey != nobody [
    ask prey [ die ]
    set energy energy + wolf-gain-from-food
  ]
end
```

# The case: Wolf and Sheep

**PATCHES procedures**



## HOW IT WORKS

- Wolves and sheep wander randomly around the landscape, taking one step at a time.
- Each step costs sheep energy.
- Sheep replenish their energy by eating grass.
- Sheep eat grass if it is available where they are.
- Sheep die when their energy falls below zero because they have not found enough grass.
- Each step costs wolves energy.
- Wolves replenish their energy by eating sheep.
- Wolves eat a sheep if one is available where they are.
- Wolves die when their energy falls below zero because they have not found enough sheep.
- Each surviving wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten, it regrows after a delay controlled by the grass-regrowth-time parameter.

```
to regrow-grass
  if not food-available? [
    ifelse countdown <= 0 [
      set food-available? true
      set pcolor green
      set countdown grass-regrowth-time
    ] [
      set countdown countdown - 1
    ]
  ]
end
```

# The case: Wolf and Sheep

using
REPORTERS



**HOW IT WORKS**

- Wolves and sheep wander randomly around the landscape. The take one “step” at a time.
- Each “step” costs the sheep energy.
- Sheep’s energy can be replenish when they eat grass.
- The sheep eat grass if available.
- Sheep die when they run out of energy. They couldn’t find enough grass.
- Each “step” costs the wolves energy.
- Wolves’ energy can be replenish when they eat sheep.
- The wolves look for sheep to prey on.
- Wolves die when they run out of energy. They couldn’t find enough sheep.
- To allow the population to continue, each wolf or sheep has a fixed probability of reproducing at each time step.
- Once grass is eaten it will regrow after a fixed amount of time.

You will get a VALUE when you call a “reporter”

```
;;;;REPORTERS
to-report land_with_grass
  report 100 * count patches with[pcolor = green] / count patches
end
```

initial-number-sheep 10

initial-number-wolves 20

grass-regrowth-time 200

setup go

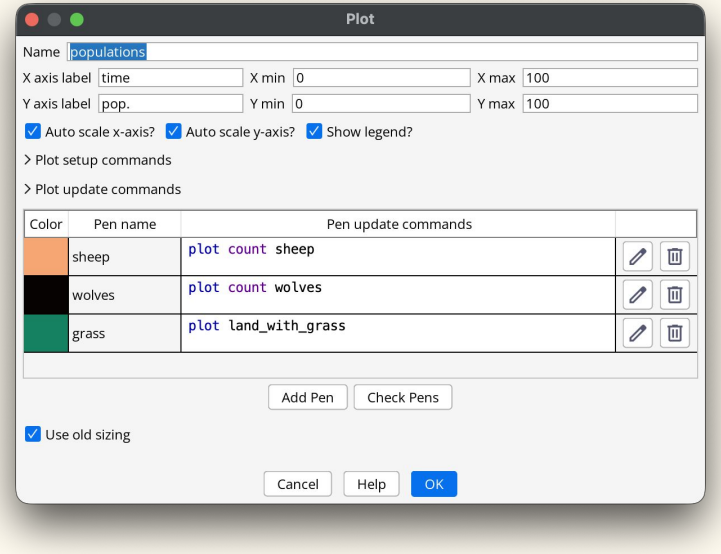
sheep-reproduce-probability 7 %

wolf-reproduce-probability 11 %

| sheep | wolves | land_with_grass |
|---|---|---|
| 10 | 20 | 49.98 |




# You will get a VALUE when you call a "reporter"






```
to-report land_with_grass
  report 100 * count patches with[pcolor = green] / count patches
end
```

# Scenario Analysis

## USING BEHAVIORSPACE: THE 100/100 AND 400/400 EXPERIMENTS

**INPUT variables:**

Vary `sheep-reproduce-probability` from 4 to 20, in increments of 2.
Vary `wolf-reproduce-probability` from 4 to 20, in increments of 2.
Vary `grass-regrowth-time` from 150 to 250, in increments of 25.

**Fixed variables** (set in Setup commands before calling `setup`):

For the **100/100** experiment: set `initial-number-sheep` and `initial-number-wolves` to 100.
For the **400/400** experiment: set both to 400.

**OUTPUT metrics:**

`land_with_grass`, `count sheep`, and `count wolves`.

**Repetitions:** 3.
**Go commands:** go.
**Run metrics every step:** Off (record only final values).
**Run metrics when:** Leave blank.

**STOP condition:**

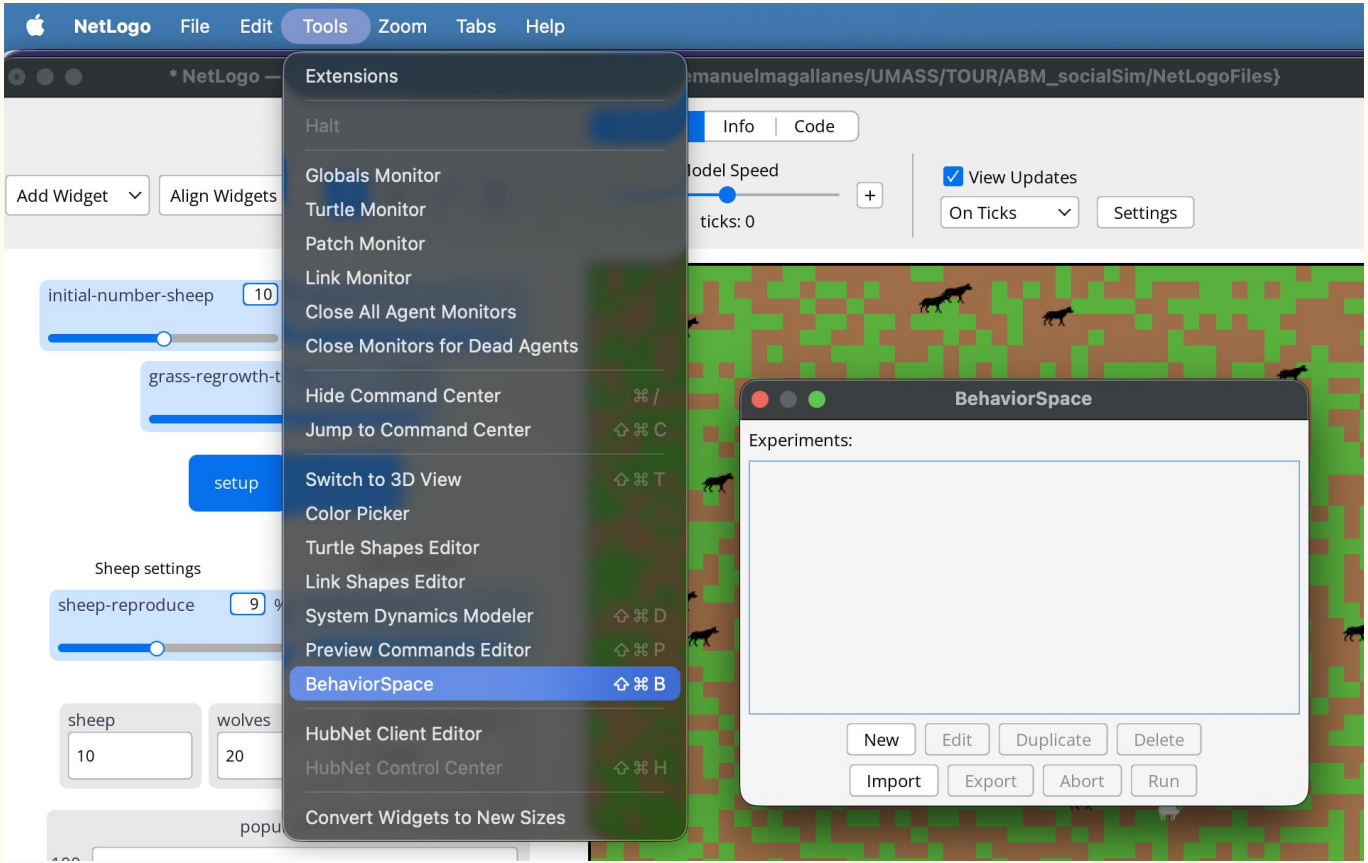
```
(land_with_grass = 0) or (not any? turtles)
```

**Time limit:** 1500.

When each experiment finishes, export its results table as `experiment_100_100-table.csv` or `experiment_400_400-table.csv`. These files will be uploaded to GitHub for analysis using Python in Colab.

**IMPORTANT FOR THE HOMEWORK**

# Scenario Analysis

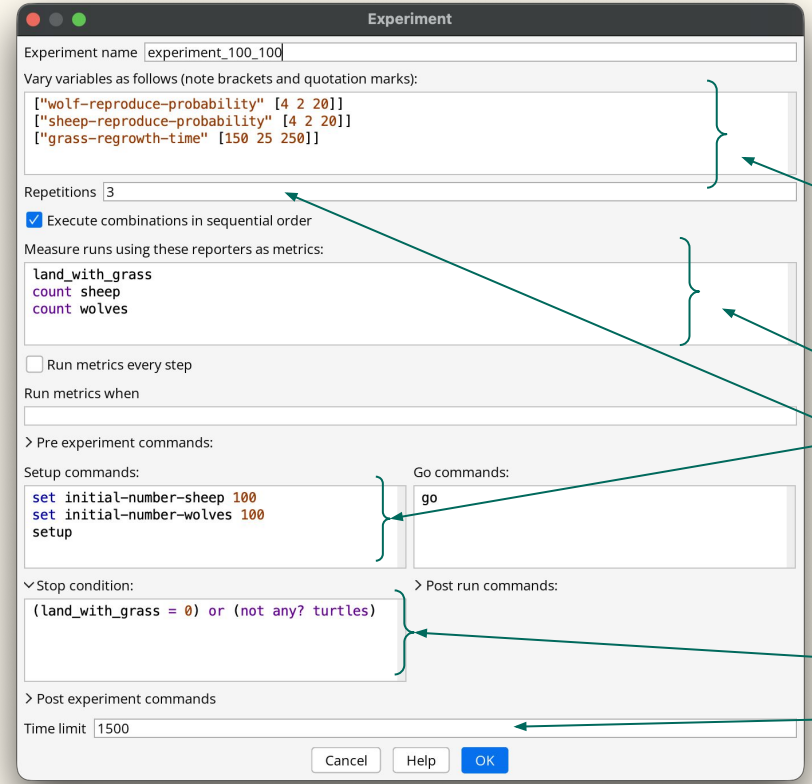



Using:

Behavior Space

# Scenario Analysis

Using:
Behavior
Space




## USING BEHAVIORSPACE: THE 100/100 AND 400/400 EXPERIMENTS

**INPUT variables:**

Vary `sheep-reproduce-probability` from 4 to 20, in increments of 2.
Vary `wolf-reproduce-probability` from 4 to 20, in increments of 2.
Vary `grass-regrowth-time` from 150 to 250, in increments of 25.

**Fixed variables** (set in Setup commands before calling `setup`):

For the **100/100** experiment: set `initial-number-sheep` and `initial-number-wolves` to 100.
For the **400/400** experiment: set both to 400.

**OUTPUT metrics:**

`land_with_grass`, `count sheep`, and `count wolves`.

**Repetitions:** 3.
**Go commands:** go.
**Run metrics every step:** Off (record only final values).
**Run metrics when:** Leave blank.

**STOP condition:**

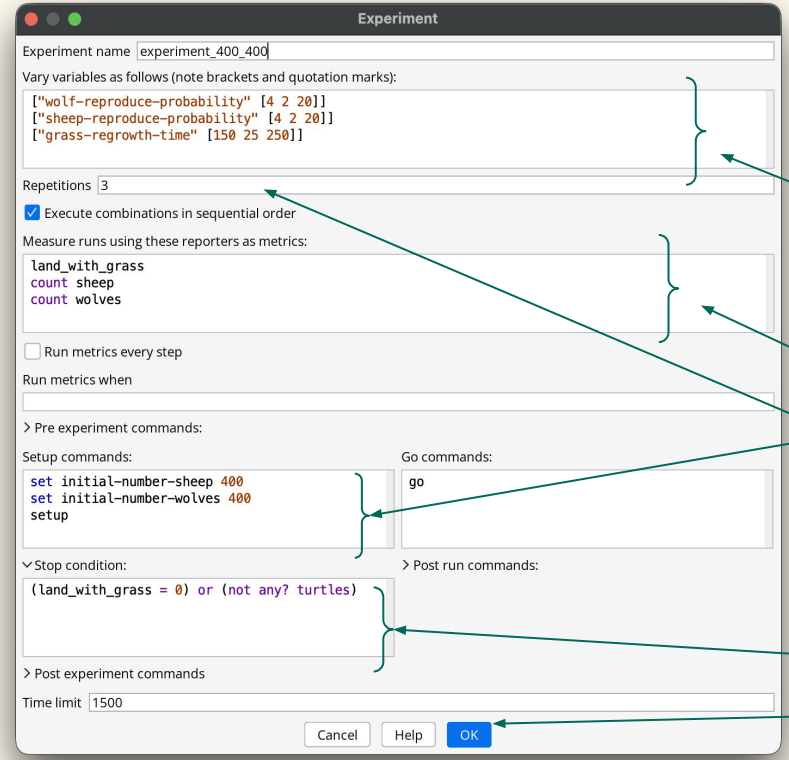
```
(land_with_grass = 0) or (not any? turtles)
```

**Time limit:** 1500.

When each experiment finishes, export its results table as `experiment_100_100-table.csv` or `experiment_400_400-table.csv`. These files will be uploaded to GitHub for analysis using Python in Colab.

# Scenario Analysis

Using:
Behavior
Space




## USING BEHAVIORSPACE: THE 100/100 AND 400/400 EXPERIMENTS

**INPUT variables:**

Vary `sheep-reproduce-probability` from 4 to 20, in increments of 2.
Vary `wolf-reproduce-probability` from 4 to 20, in increments of 2.
Vary `grass-regrowth-time` from 150 to 250, in increments of 25.

**Fixed variables** (set in Setup commands before calling `setup`):

For the **100/100** experiment: set `initial-number-sheep` and `initial-number-wolves` to 100.
For the **400/400** experiment: set both to 400.

**OUTPUT metrics:**

`land_with_grass`, `count sheep`, and `count wolves`.

**Repetitions:** 3.
**Go commands:** go.
**Run metrics every step:** Off (record only final values).
**Run metrics when:** Leave blank.

**STOP condition:**

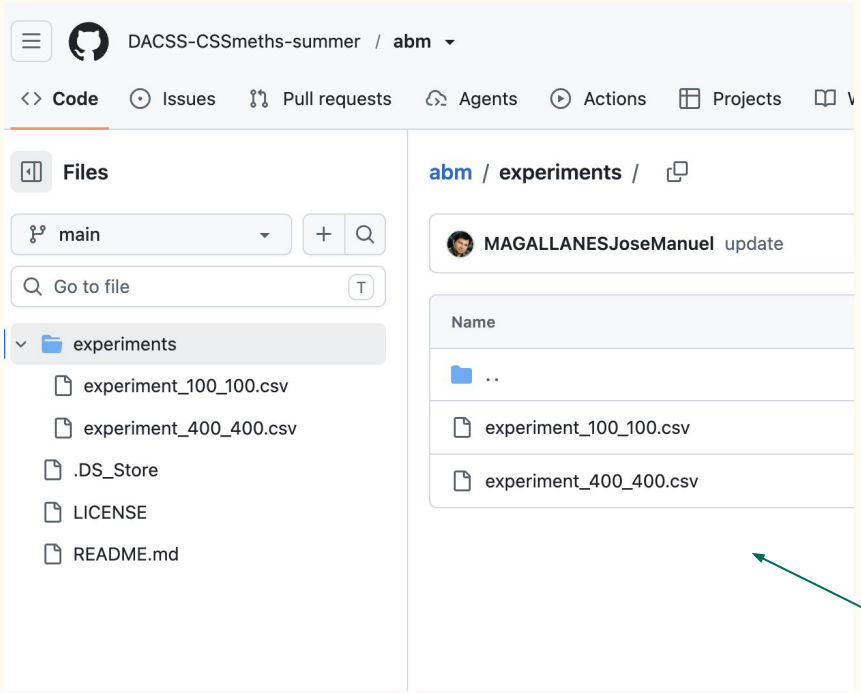
```
(land_with_grass = 0) or (not any? turtles)
```

**Time limit:** 1500.

When each experiment finishes, export its results table as `experiment_100_100-table.csv` or `experiment_400_400-table.csv`. These files will be uploaded to GitHub for analysis using Python in Colab.

# Scenario Analysis

Using:
Behavior
Space




## USING BEHAVIORSPACE: THE 100/100 AND 400/400 EXPERIMENTS

**INPUT variables:**

Vary `sheep-reproduce-probability` from 4 to 20, in increments of 2.
Vary `wolf-reproduce-probability` from 4 to 20, in increments of 2.
Vary `grass-regrowth-time` from 150 to 250, in increments of 25.

**Fixed variables** (set in Setup commands before calling `setup`):

For the **100/100** experiment: set `initial-number-sheep` and `initial-number-wolves` to 100.
For the **400/400** experiment: set both to 400.

**OUTPUT metrics:**

`land_with_grass`, `count sheep`, and `count wolves`.

**Repetitions:** 3.
**Go commands:** go.
**Run metrics every step:** Off (record only final values).
**Run metrics when:** Leave blank.

**STOP condition:**

```
(land_with_grass = 0) or (not any? turtles)
```

**Time limit:** 1500.

When each experiment finishes, export its results table as `experiment_100_100-table.csv` or `experiment_400_400-table.csv`. These files will be uploaded to GitHub for analysis using Python in Colab.

# Scenario Analysis

**Results from the Behavior Space experiments will need to be analyzed OUTSIDE NetLogo**

VISIT THE **COLAB** NOTEBOOK LINK PROVIDED

## Wolf Sheep Predation — Comparing Two BehaviorSpace Runs

Comparing **100 sheep / 100 wolves** vs **400 sheep / 400 wolves** — same model code (unmodified), same parameter grid ( `sheep-reproduce` , `wolf-reproduce` , `grass-regrowth-time` ), same time limit (1500 ticks).

### Data links:

[1] ✓ 0s

```
bs100link='https://github.com/DACSS-CSSmeths-summer/abm/raw/main/Wolf_Sheep_Predation_experiment_100_100-table.csv'
bs400link='https://github.com/DACSS-CSSmeths-summer/abm/raw/main/Wolf_Sheep_Predation_experiment_400_400-table.csv'
```

**You will bring another file, and compare it to ONE of these**

[2] ✓ 1s

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option("display.max_columns", None)
```

### 1.- Load the data

BehaviorSpace table exports start with 6 lines of metadata (model name, timestamp, world dimensions) before the actual header row, so we skip them with `skiprows=6` . We also rename the columns to something easier to work with in code — the raw column names come out as things like `[run number]` and `sheep-reproduce` , which are awkward to reference in pandas.

[3] ✓ 0s

```
def read_bs_table(path):
    df = pd.read_csv(path, skiprows=6)
    df.columns = ["run_number", "sheep_reproduce", "wolf_reproduce",
                  "grass_regrowth_time", "step",
                  "land_with_grass", "count_sheep", "count_wolves"]
    return df

exp_100 = read_bs_table(bs100link)
exp_400 = read_bs_table(bs400link)
```